

|                    |                      |
|--------------------|----------------------|
| <b>Challenge:</b>  | Fragmented Challenge |
| <b>Category:</b>   | Encoding             |
| <b>Difficulty:</b> | Easy                 |
| <b>Tools Used:</b> | Python IDE           |

## Summary

The challenge provides a string that doesn't match standard encodings but clearly follows a pattern. It hints that the data has been reduced by combining multiple pieces into smaller units. The goal is to recognize this encoding and reverse the process to recover the original hidden message.

In brief, the procedure involves grouping the string into fixed-size units, mapping each character to its corresponding index from the given character set, and combining these values to reconstruct the original numerical representation. A small offset is then corrected, after which each unit is expanded back into its original byte form. Repeating this process for the entire string ultimately reveals the hidden message.

## Problem Statement

The data is structured, but unfamiliar. It does not match any standard encoding, yet it appears too deliberate to be random. The signal seems compressed—as if multiple fragments were folded into fewer units. Its size is smaller than expected for the information it might contain. Whatever transformation was applied, it was not meant to make the data unreadable—only less obvious. Reconstruct the original message.

## Steps to Solve

1. Upon opening the challenge.txt file, the following encoded string is obtained:  
EN5HV@Gh8G57HtpDdmHNwGdNG9zGRmGFAEY6GtTF8!DJrFCE
2. Initial observation shows that the string does not match standard encodings such as Base64. As suggested by **Hint 1**, the signal uses a limited and consistent set of characters, indicating a structured format. This implies that the focus should be on analysing its internal pattern rather than forcing it into a known encoding scheme.
3. The data appears shorter than expected for meaningful output. According to **Hint 2**, this suggests that each encoded unit may represent more than one byte, indicating a compact encoding mechanism.
4. Based on its structure, the string is divided into groups of three characters, where each group forms a single encoded unit.
5. Using the predefined character set (mapped from index 0 to 63), each character in a group is converted into its corresponding numerical index ( $c_1$ ,  $c_2$ ,  $c_3$ ).
6. As guided by **Hint 3**, each group is converted back into a number using the formula:

$$\text{Val} = c_1 \times 64^2 + c_2 \times 64 + c_3$$

7. The resulting value appears slightly offset, confirming the hint that a small adjustment is required. The correction is applied by subtracting a constant offset:  $Val = Val - 7$
8. As indicated by **Hint 4**, the transformation does not preserve size in a one-to-one manner. Instead, each encoded unit expands into exactly two bytes after decoding.
9. Therefore, the adjusted integer is split into two bytes and appended to reconstruct the original data stream.
10. Repeating this process for all groups successfully decodes the string and reveals the final flag.

```
File Edit Format Run Options Window Help
CHARSET = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@"
BASE = len(CHARSET)
OFFSET = 7

data = "EN5HV@Gh8G57HtpDdmHNwGdNG9zGRmGFAEY6GtTF8!DJrFCE"

decoded = bytearray()

for i in range(0, len(data), 3):
    c1 = CHARSET.index(data[i])
    c2 = CHARSET.index(data[i+1])
    c3 = CHARSET.index(data[i+2])

    val = c1 * (BASE**2) + c2 * BASE + c3
    val -= OFFSET

    decoded.append((val >> 8) & 0xFF)
    decoded.append(val & 0xFF)

print(decoded.decode(errors="ignore"))
```

Flag: Cruxhunt{b7\_sigFold\_a9F3kL\_72dP}